Seatwork No. 6.2

Seatwork 6.2: Built-in Functions

Course Code: CPE 007	**Program: Computer Engineering**
Course Title: Programming logic and Design	**Date Performed:October 22, 2026**
Section: CPE11S2	**Date Submitted:October 22, 2026**
Name: Jose. Redge Jansen A.	**Instructor: Sir Noel Jason Lusung**

1. Objective(s)

This activity aims to develop programming skills using functions.

2. Intended Learning Outcomes

1.) Work with files.
2.) Read from a file.
3.) Write to a file
4.) Process sample data
5.) Understand the concept of the loop over file content

3. Materials and Equipment

Internet connection
Personal Computer

4. Output

1. Create a program that reads floating-point numbers line-by-line from a text file. If non-numeric characters are encountered on a line, log a warning, clear the error, and skip to the next line without crashing.
Answer:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

int main()
{
   string filename;
   cout << "Enter the file name: ";
   getline(cin, filename);

   ifstream file(filename);
   if (!file)
   {
      cerr << "Error: could not open file \"" << filename << "\"\n";
      return 1;
   }

   string line;
   int lineNumber = 0, validCount = 0, skippedCount = 0;
   double sum = 0.0;

   while (getline(file, line))
   {
      lineNumber++;
```

```
        // Skip blank lines quietly
        if (line.find_first_not_of(" \t\r") == string::npos)
            continue;

        istringstream iss(line);
        double value;

        // Try to read a number, then make sure nothing but whitespace follows
        if (iss >> value && (iss >> ws).eof())
        {
            cout << "Line " << lineNumber << ": " << value << "\n";
            sum += value;
            validCount++;
        }
        else
        {
            cerr << "Warning: line " << lineNumber
                 << " contains non-numeric data (\"" << line
                 << "\"). Skipping.\n";
            iss.clear();          // clear the error state
            skippedCount++;       // move on to the next line
        }
    }

    file.close();

    cout << "\nValid numbers read: " << validCount << "\n";
    cout << "Lines skipped:      " << skippedCount << "\n";
    cout << "Sum of valid numbers: " << sum << "\n";

    return 0;
}
```

Screenshot:

```
Enter the file name: Numbers.txt
Line 1: 3.14159
Line 2: 8248
Warning: line 3 contains non-numeric data ("12 24 2026"). Skipping.

Valid numbers read: 2
Lines skipped:      1
Sum of valid numbers: 8251.14
```

2. Write a program that opens a text file containing lines of mixed numbers and words. Read through the file line by line, print valid numbers, and handle/skip lines containing invalid text data without halting execution.

Answer:

```
#include <iostream>
#include <fstream>
#include <sstream>
```

```
#include <string>
using namespace std;

int main()
{
    string filename;
    cout << "Enter the file name (with extension, e.g. data.txt): ";
    getline(cin, filename);

    ifstream file(filename.c_str());
    if (!file)
    {
        cerr << "Error: could not open file \"" << filename << "\"\n";
        cerr << "Check the file name, extension, and folder.\n";
        return 1;
    }

    string line;
    int lineNumber = 0, validCount = 0, skippedCount = 0;

    cout << "\nValid numbers:\n";

    while (getline(file, line))
    {
        lineNumber++;

        // Ignore blank lines
        if (line.find_first_not_of(" \t\r") == string::npos)
            continue;

        istringstream iss(line);
        double number;

        // Valid only if a number is read AND nothing else follows it
        if (iss >> number && (iss >> ws).eof())
        {
            cout << "  Line " << lineNumber << ": " << number << "\n";
            validCount++;
        }
        else
        {
            cout << "  Line " << lineNumber << ": skipped (invalid data: \""
                 << line << "\")\n";
            iss.clear();
            skippedCount++;
        }
    }

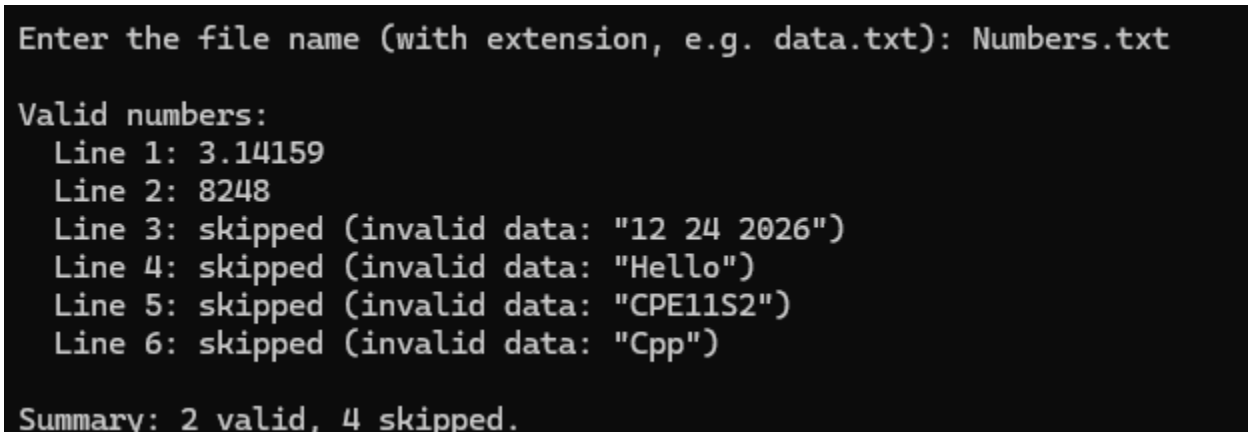
    file.close();

    cout << "\nSummary: " << validCount << " valid, "
```

```
        << skippedCount << " skipped.\n";

    return 0;
}
```

Screenshot:



5. Conclusion

In these activities, I learned how to work with files in C++ by using `ifstream` and `getline` to read a text file line by line. I used an `istringstream` to tell valid numbers apart from invalid text, and `clear()` to reset the error state so the program could skip bad lines instead of crashing. Overall, these activities strengthened my understanding of file input, loops, and error handling.

6. Rubric